

Relatório Final - Harness e Arquitetura na Prática

1. Autonomia e guardrail (Etapa 1)

Duas tarefas de mesmo porte foram feitas em modos diferentes: `validador_placa` em **plan mode** (exploração, plano escrito, aprovação antes de codar) e `validador_cep` em **execução direta**. A implementação em si levou tempo parecido nos dois casos (~20s vs ~17s); a diferença real foi controle: no plan mode, um engano de escopo (quase duplicar um CPF que já existia em outra branch) foi pego *antes* de qualquer arquivo ser criado - no fluxo direto, esse tipo de checagem tende a ser pulado. Guardrail implementado: hook `PreToolUse` que bloqueia `pip install` de pacote não documentado em `docs/dependencias-aprovadas.md`, aplicando a regra do `CLAUDE.md`. Testado ao vivo (bloqueou `pip install requests`, deixou passar `pytest` aprovado) e revelou, no processo, um bug real no regex de parsing de argumentos de shell (corrigido). Detalhes: `docs/etapa1-* .md`.

2. TDD e enforcement (Etapa 2)

Ciclo Red-Green-Refactor real em `validar_cartao_credito` (3 commits distintos: teste falho, implementação, refactor). Investigação do plugin **tdd-guard** (instalado de verdade via marketplace oficial do Claude Code): o reporter `pytest` funciona e grava resultados reais, mas o hook de bloqueio usa por padrão um cliente de IA (sdk) que exige credencial própria - sem ela, ele **falha aberto** (permite tudo, silenciosamente) em vez de bloquear. Comparação com/sem TDD: `validador_hora`, implementado direto sem teste antes, passou despercebido por 3 casos de borda (formato de 2 dígitos, dígitos unicode) só descobertos ao escrever teste depois - o cartão de crédito, feito em TDD, não teve nenhum gap equivalente. Ver `docs/etapa2-tdd.md`.

3. Checkpoint humano (Etapa 3)

Checkpoint definido: qualquer mudança de configuração global do Claude Code (plugins, hooks fora do repositório) para a execução até aprovação explícita - porque o git não protege contra esse tipo de mudança. Simulação real (não encenada): a instalação do `tdd-guard` só rodou depois de uma pergunta direta ao usuário, que aprovou. Papel humano assumido: aprovador final de mudanças fora do escopo versionado. Log completo da sessão em `docs/sessao-log.md`.

4-5. Decisão arquitetural, ADR e diagrama (Etapas 4 e 5)

Análise do código real mostrou zero acoplamento entre os 6 validadores (nenhum importa outro) e alta coesão - 217 linhas de produção, abaixo de qualquer limiar que justifique extrair serviço ou criar subpacotes. Decisão: manter como está (ADR 0002), com uma duplicação pontual (checar tipo + normalizar string, repetida em 4 arquivos) registrada como melhoria futura, não como problema estrutural. Dois diagramas C4 de contêiner foram gerados: um com prompt curto (genérico, poderia ser qualquer projeto Python) e um com contexto da própria atividade (mostra hooks, `tdd-guard` e Claude Code como elementos da arquitetura). O segundo comunica melhor porque representa o *processo* - que era o objeto real desta atividade, não só o produto.

6. Vá além - agentes em paralelo (Etapa 6, opção C)

Dois agentes rodaram em `worktrees` isolados, cada um criando um validador novo e disputando de propósito a mesma seção do README. O merge do conflito foi trivial. O achado sério foi outro: os `worktrees` nasceram da branch `main` em vez da branch de trabalho da atividade, deixando hooks e validadores de fora - isso travou o Bash de um agente inteiro (hook com caminho relativo, arquivo ausente) e, pior, travou também a sessão principal depois que ela entrou manualmente num desses `worktrees`: nem o comando `cd` de recuperação rodava, porque o próprio hook bloqueava antes de qualquer comando. A saída só veio delegando a limpeza a um agente novo, com `cwd` limpo. Aprendizado principal: isolamento por `worktree` não isola contra configuração que só existe em outra branch - e hooks com caminho relativo são frágeis a exatamente esse tipo de mudança de contexto.

Dificuldade real enfrentada

A mais séria foi técnica: o `cwd` da sessão principal ficou preso dentro de um `worktree` cujo hook estava quebrado, e como o hook roda *antes* de qualquer comando Bash (inclusive o `cd` que resolveria o problema), a sessão ficou momentaneamente sem conseguir usar Bash de jeito nenhum - só resolvido delegando para um agente novo, sem esse estado contaminado.